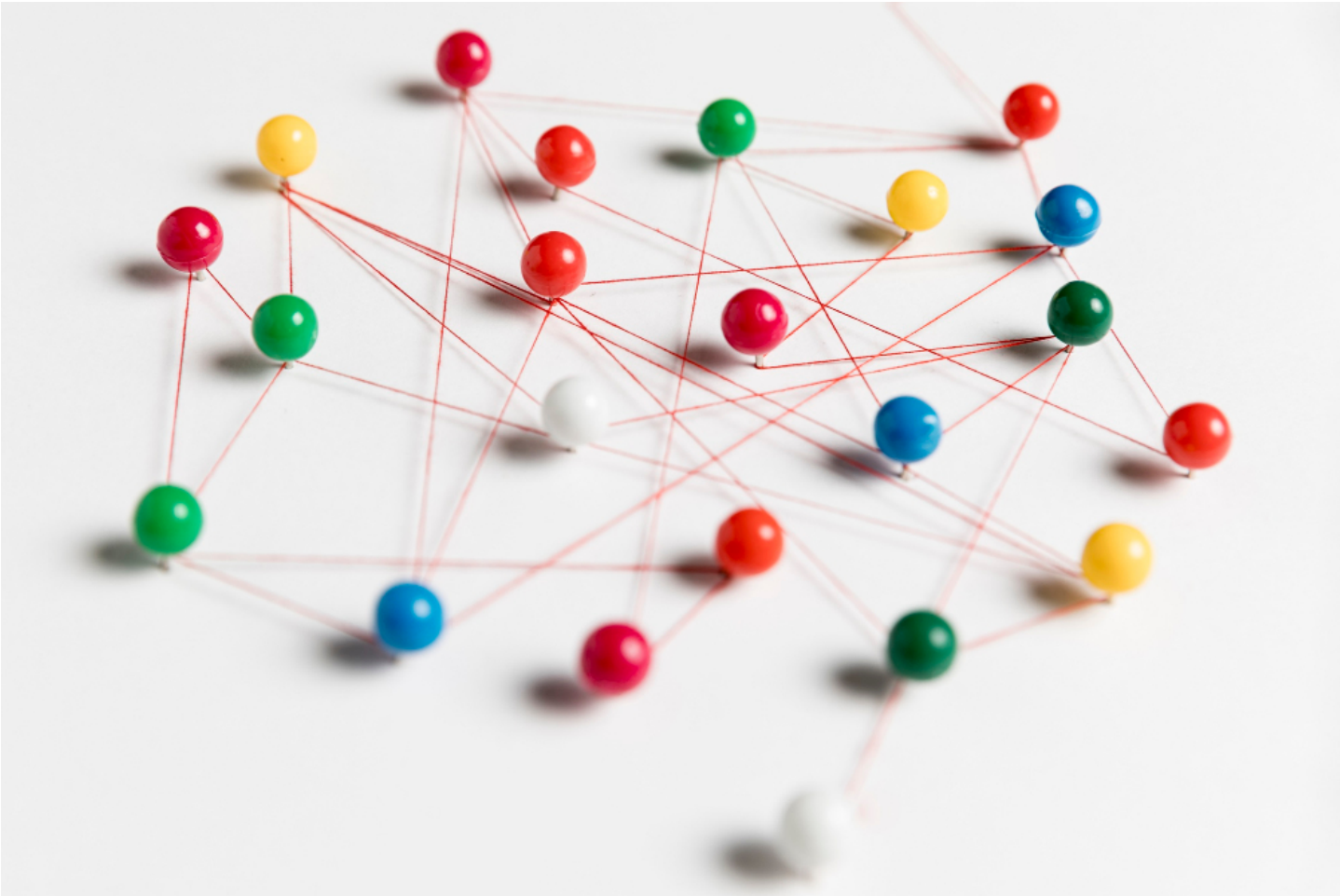
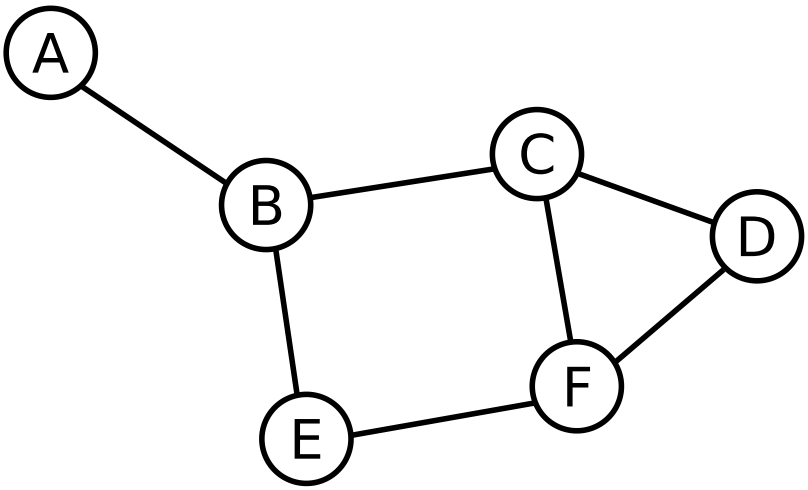


Image: Freepik.com

Graphs

What is a graph?

Graphs come from graph theory in discrete mathematics and refers to a group of objects or nodes, where at least some of them are connected somehow to each other.

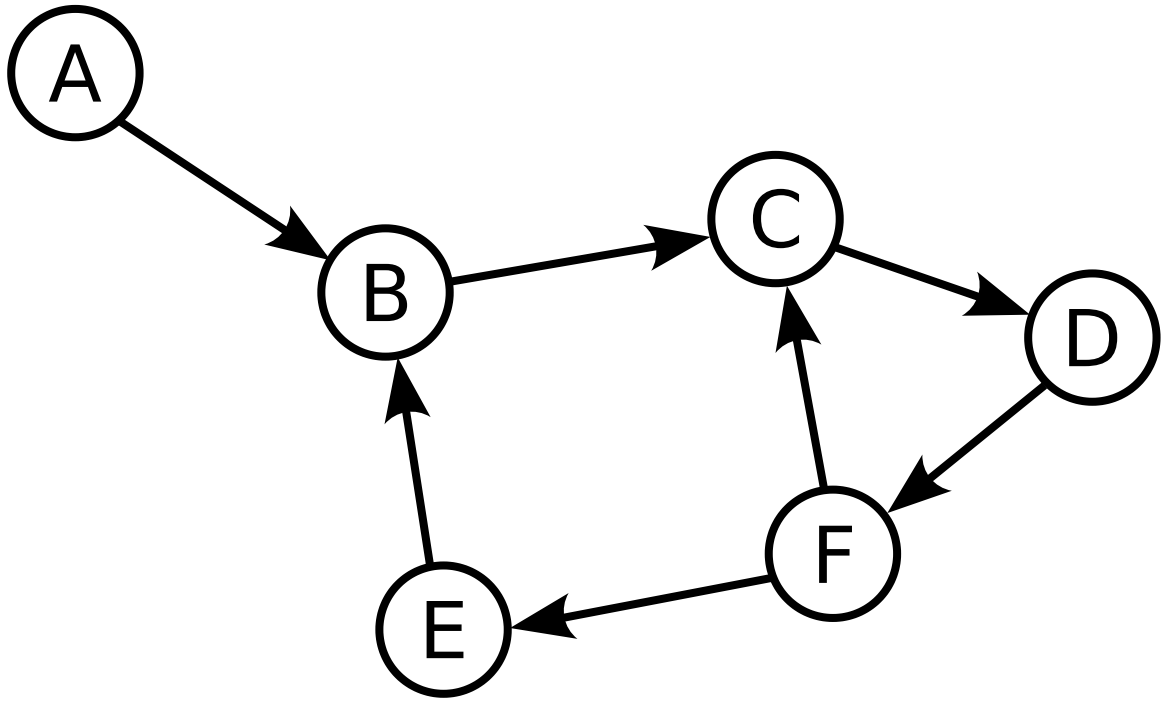


The **nodes** or **vertices** (singular: **vertex**) are the objects being connected. They are usually represented as circles, dots or similar figures. The connections between vertices are called **edges** and they are usually represented as lines or arrows between the vertices. It is possible for a vertex to be connected with itself in a **self-loop**. An edge is said to be **incident** to a vertex if the vertex is one of the edge's endpoints. Vertices can have several edges incident to them. The number of edges incident to a vertex is the **degree** of the vertex (Loops count as well). Vertices are **adjacent** if there is an edge between them. And given that the vertices are connected, it is possible to create a route or **path** between two vertices that are not adjacent.

Directed vs undirected graphs

When talking about edges, it was mentioned that they can be represented by lines or arrows. This also shows the difference between **directed** and **undirected** graphs. In directed graphs, edges have a direction. They go from one vertex to another vertex. By contrast, in undirected graphs, connections have no direction. There are also **mixed** graphs that contain directed and undirected edges, but we are going to ignore them in this course.

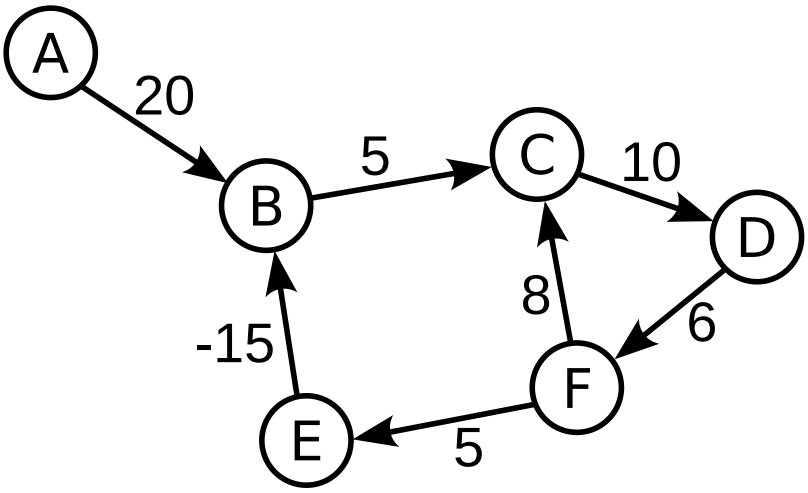
In the following directed graph, there is no way to reach vertex A from the other vertices.



Given the nature of directed graphs, it is not possible to guarantee that given two vertices, is always going to be a path connecting them. At least in both directions. In the image above it is possible to reach C from A, but it's not possible to reach A from C.

Weighted graphs

The edges can have a so called **weight** that indicates some value related to the edge. It can represent different things. It can be a distance, it can be a time to travel, it can be a price, etc... It could be whatever numeric property related to the edge. It depends on what the graph represents. These weights are important as many of the actions we want to perform on graphs are based on them. For example, what is the shortest path between two given vertices? Or the cheapest? Or the fastest? Again, the possibilities are almost unlimited.



However, this have a consequence for us, as this means the edges have some intrinsic information that has to be stored. So probably is better to treat edges as objects (same as vertices) so we can store information about them.

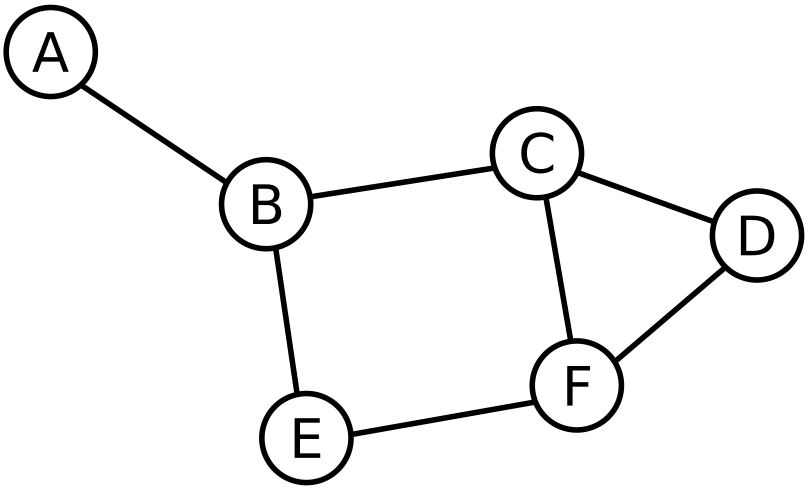
Data structures for graphs

Graphs need to be represented in some "formal" way (not just by drawing), so we can create a data structure and write functions and programs that use them. There are four main forms of organizing a graph:

Edge list

It is just an unordered list of all edges. This would be sufficient to define a graph, but it's not efficient. For example, it could of time complexity $O(n)$ to find all edges incident to a vertex.

Given the example we have been using:



a possible Python representation could be with a Set:

```
graph = set((C, D), (E, F), (A, B), (B, E), (C, F), (D, F), (B, C))
```

Note

The identifiers showed in the example (A, B, C, D, E and F) are to be considered identifiers for Vertex objects (Vertex instances or variables). The edges have been represented with tuples, but in the actual implementation, we will create an Edge class.

Adjacency list

Another way of representing a graph is with an **adjacency list**. As the name suggests, a list is put together with the lists of edges adjacent to each vertex.

The adjacency list could be represented like:

```
graph = [
    [(A, B)],
    [(A, B), (B, C), (B, E)],
    [(B, C), (C, D), (C, F)],
    [(C, D), (D, F)],
    [(B, E), (E, F)],
    [(C, F), (D, F), (E, F)],
]
```

or even with a dict:

```
graph = {
    A: [(A, B)],
    B: [(A, B), (B, C), (B, E)],
    C: [(B, C), (C, D), (C, F)],
    D: [(C, D), (D, F)],
    E: [(B, E), (E, F)],
    F: [(C, F), (D, F), (E, F)],
}
```

This way is easier to find the edges of a given vertex. But if we want to find an edge object given its endpoints, we still have to traverse the secondary list.

Adjacency map

The **adjacency map** is similar to the adjacency list, but the secondary container is organized as a map/dictionary instead of a list.

For example:

```
graph = {
    A: {B: (A, B)},
    B: {A: (A, B), C: (B, C), E: (B, E)},
    C: {B: (B, C), D: (C, D), F: (C, F)},
    D: {C: (C, D), F: (D, F)},
    E: {B: (B, E), F: (E, F)},
    F: {C: (C, F), D: (D, F), E: (E, F)},
}
```

This kind of structure has some advantages, like the fact that it should be possible to get and edge in $O(1)$, as it is possible to do `graph[u][v]`, being `u` and `v` the endpoints vertices. Dictionaries use hashing, and as we saw in the Hashing chapter, it is possible, depending on the implementation, that the look up takes more time. So the time complexity is actually $O(1)$ of **expected** time (but the actual time of a single look up could take more.)

This is probably the best structure to represent a graph, although it is always case dependent. But generally, the adjacency map offers good complexity times at a relative low space requirement.

Adjacency matrix

The final way of representing a graph is with an **adjacency matrix**. A matrix is a two-dimensional array and in Python is normally represented with a list of lists (all sublists of the same length). The idea is to use the vertices as column and row headers and use 1 and 0's to show if those two vertices are connected (if there is an edge between them).

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	1	0	1	0	1	0
C	0	1	0	1	0	1
D	0	0	1	0	0	1
E	0	1	0	0	0	1
F	0	0	1	1	1	0

And this can be translated to python like:

```
graph = [
    [0, 1, 0, 0, 0, 0],
    [1, 0, 1, 0, 1, 0],
    [0, 1, 0, 1, 0, 1],
    [0, 0, 1, 0, 0, 1],
    [0, 1, 0, 0, 0, 1],
    [0, 0, 1, 1, 1, 0]
]
```

In this case, the vertices should be used like numeric indexes, i.e. to be able to use this structure as `graph[u][v]`, then `u` and `v` should be positive integers. Vertices should be numbered from 0 to $n-1$ (being n the number of vertices)

The adjacency matrix can also be created from the adjacency list or adjacency map:

```
# Adjacency list:
all_vertices = graph_adjacency_list.keys()
graph_adjacency_matrix = [[int(vertex in adjacents) for vertex in all_vertices] for
adjacents in graph_al.values()]

# Alternative for adjacency list:
# graph_adjacency_matrix = [[int(vertex in graph_al[node_row]) for vertex in all_vertices ]
for node_row in all_vertices]

# Adjacency map:
all_vertices = adjacency_map.keys()
graph = [[int(bool(adjacency_map[u].get(v))) for v in all_vertices] for u in all_vertices]
```

Or if you prefer in a more iterative way (with adjacency map):

```
all_vertices = adjacency_map.keys()
adjacency_matrix = []
for u in all_vertices:
    row = []
    adjacents = adjacency_map[u].keys()
    for v in all_vertices:
        row.append(int(v in adjacents))
    adjacency_matrix.append(row)
```

The adjacency matrix offers the best access time, as it is guaranteed to be $O(1)$, as this structure uses arrays. But other actions can be less efficient and adding or deleting vertices forces the matrix to be resized. Space requirements are little bit worse than other structures.

Python implementation

Let's try to implement some structures. Starting with a vertex. The vertex has to hold some value. We add the `__repr__()` method to facilitate the debugging. In this case, we also add a `__hash__()` method so the structure can work as a key for a dictionary.

```
class Vertex:
    def __init__(self, value):
        self._value = value

    def __repr__(self):
        return f'<Vertex: {self._value}>'

    def __hash__(self):
        return hash(id(self))
```

Then an Edge. As we said before, it could be good to implement the edges as objects, as they can have their own values or properties. As we are working with undirected graphs, we don't take into account directions. As with the previous object, we implement the `__repr__()` and `__hash__()` methods. It's also implemented a few basic methods more, to help with the edges handling.

```
class Edge:
    def __init__(self, u, v, x):
        self._first = u
        self._second = v
        self._value = x

    def __repr__(self):
        return f'<Edge ({self._value}): {self._first} --> {self._second}>'

    def endpoints(self):
        return (self._first, self._second)

    def opposite(self, v):
        return self._second if v is self._first else self._first

    def value(self):
        return self._value

    def __hash__(self):
        return hash( (self._first, self._second) )
```

And finally, let's implement the graph itself.

```python
class Graph:
    def __init__(self, adj_map = None):
        if adj_map:
            self._adj_map = adj_map
        else:
            self._adj_map = {}

    def get_vertices(self):
        return self._adj_map.keys()

    def get_edges(self):
        """Return a set of all edges of the graph."""
        result = set()
        for secondary_map in self._adj_map.values():
            result.update(secondary_map.values())
        return result

    def get_edge(self, u, v):
        """
        Returns the edge from u to v, or None if not adjacents.
        """
        return self._adj_map[u].get(v)

    def degree(self, u):
        """
        Returns the number of edges incident to vertex u
        """
        return len(self._adj_map[u])

    def get_adjacent_vertices(self, u):
        """
        Return a list of the adjacent vertices of a given vertex
        """
        return list(self._adj_map[u].keys())

    def get_incident_edges(self, u):
        """
        Returns edges incident to vertex u
        """
        return list(self._adj_map[u].values())

    def add_vertex(self, value):
        vertex = Vertex(value)
        self._adj_map[vertex] = {}
        return vertex

    def add_edge(self, u, v, x=None):
```

```
        edge = Edge(u, v, x)
        self._adj_map[u][v] = edge
        self._adj_map[v][u] = edge

    def get_adj_map(self):
        return self._adj_map

    def get_adj_matrix(self):
        all_vertices = self._adj_map.keys()
        return [[int(bool(self._adj_map[u].get(v))) for v in all_vertices] for u in
all_vertices]
```

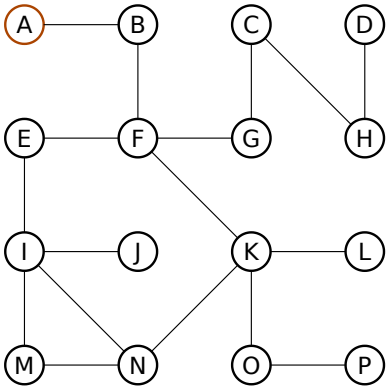
Graph traversal

Traversal of the graphs can be similar to traversing trees, and we actually have the same two strategies to achieve it: Depth-First Search and Breadth-First Search

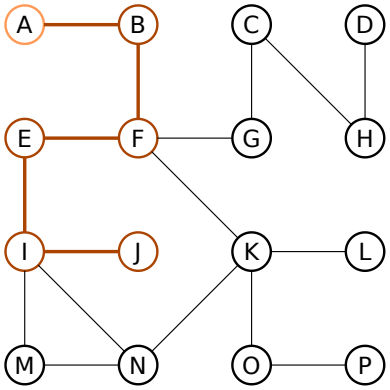
In this case the idea is to visit all vertices and edges and do it in $O(n+m)$, that is in a time proportional to their number (being n the number of vertices and m the number of edges). In graphs, traversal algorithms are a fundamental part of their use, as many things we want to achieve with graphs require traversing the graph and getting information about paths and **reachability**

Depth-First Search (DFS)

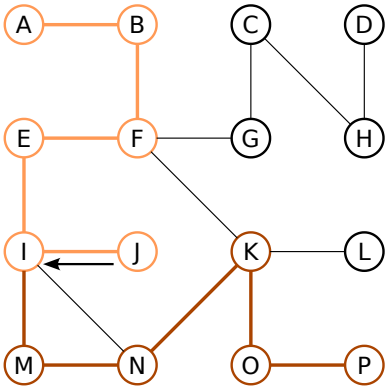
The idea with **Depth-First Search (DFS)** is the same as with trees. Starting from a given vertex, the algorithm will try to go forward until it can not advanced anymore. Meaning that it reaches a vertex that has no unvisited adjacents. In that case, it will go back to a vertex that still has unvisited adjacents and again will try the same approach. This will be repeated until there is no more vertices to visit and the algorithm backtracks to the starting point.



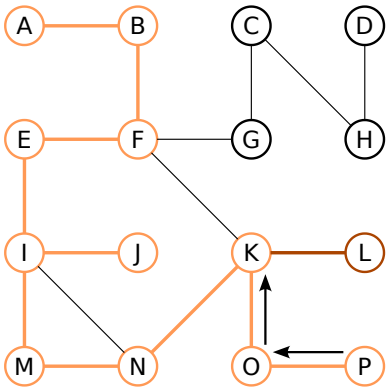
Starting state



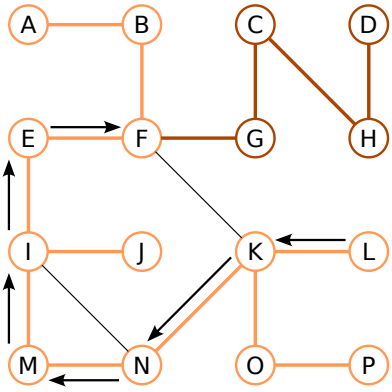
Forward from A to J



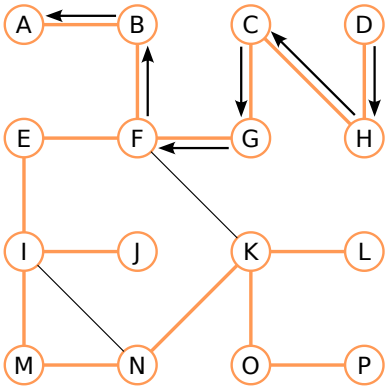
Backtrack from J to I and forward from I to P



Backtrack from P to K and forward from K to L



Backtrack from L to F and forward from F to D



Backtrack from D to A and return from DFS function

In graphs there is no Root node, so the starting point is to be decided by the user based on some criteria (what are we trying to achieve?)

The implementation of this function should be relatively easy. One popular approach is to do it recursively. This way is easy to backtrack to a vertex from where is possible to continue. The idea is to maintain a variable with the visited vertices, where we can quickly check if a vertex has been visited or not. One popular option for this is a dictionary, where the key is the vertex, and the value is the edge or the adjacent node that was used to get to this vertex.

In the main loop the function will iterate over the adjacents of the given vertex and if the adjacent has not been visited, the function will add it to the dict and make a recursive call with the adjacent vertex as the start node and the dict as second parameter.

When all the adjacents has been visited, the function can return the dictionary.

Exercise!

Applying what we have just seen, implement a Depth-First Search traversal function.

Don't continue until you have finished the exercise!

```python
def DFS(graph, u, visited=None):
    """
    Perform Depth-First Search of the undiscovered portion of the graph starting at Vertex
u.
    """
    # Function should be called without a visited dict
    # as one will be created in the first call.
    # The starting vertex will be added in that case
    if visited is None:
        visited = {u: None}
    # Traverse all adjacents of u
    for v in graph.get_adjacent_vertices(u):
        # Check if the adjacent has been visited before
        if v not in visited:
            # If not visited, add it to the dict
            visited[v] = u
            # And make the recursive call on the vertex
            DFS(graph, v, visited)
    # When the function backtracks to the start,
    # then all nodes and edges have been visited
    # and the list can be returned
    return visited
```

An iterative version can also be implemented with a Stack -like structure.

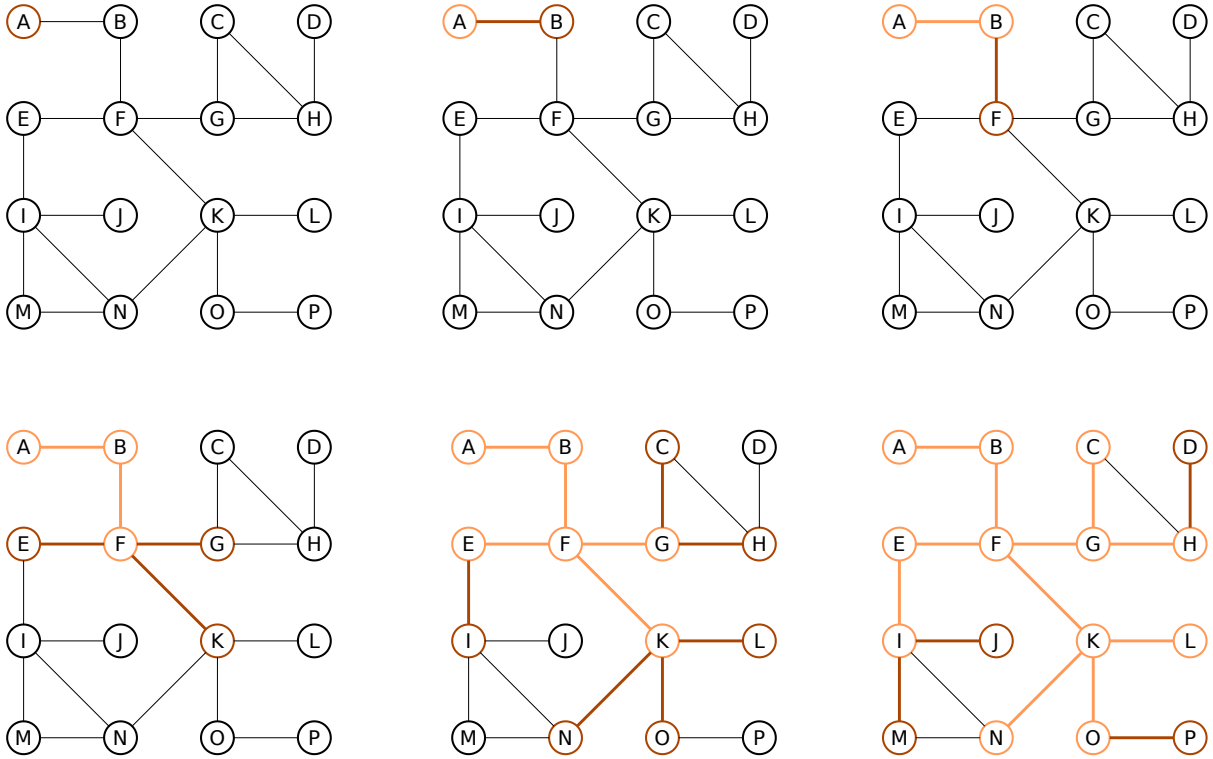
DFS is of time complexity $O(n+m)$ (being n the number of vertices reachable from starting vertex and m the number of edges incident to those vertices). Every vertex that can be reached from the starting point is reached once (the visited list ensures that vertices are not visited twice). And edges incident to those vertices are checked twice (one from each end). That is n+2m. We take only the polynomial leading factors, and that gives us the $O(n+m)$

We have to assume that `self.get_adjacents(u)` runs in $O(degree(u))$. If that would not be the case, then we can no longer maintain the time complexity of $O(n+m)$ for the whole algorithm. The line `visited[v] = u` should also run in $O(1)$, but as a dict is implemented with hashing, we know that the best complexity time is $O(1)$ **expected** time. But this is a compromise. If we want pure $O(1)$ runtime for this operation, then an array has to be used (as it has $O(1)$ access time) and vertices should be numbered beforehand.

The algorithm stores the edge from it discovers a new vertex along with the vertex. This allows to check if a vertex if reachable from the starting vertex and what would be the path to it. But there are also other problems that can be solved with traversing and DFS.

Breadth-First Search (BFS)

The other important method is **Breadth-First Search (BFS)** and it uses a different strategy. It also goes in steps starting from a given vertex, but in each step, it spreads as much as possible reaching all vertices that are reachable at that step. This creates like levels of vertices that can be reached at each step, starting from the given vertex, which would be level 0. This is the same as with trees, where the levels concept is more visually clear.



For the implementation we are going iterative this time. Same as before a dict is used to store the discovered vertices and the vertex that was used to reach to them. Two lists are used (one for the current level and a second one for the next level), but in the end, they work as a queue, so an actual queue could also be used. Aside from that, the idea is simple, there is a main loop that will be executed if the level has elements. In the main loop initialize the next level's list and traverse the vertices of the current level, adding the adjacents of such vertices if they are not visited. After traversing the vertices, just update the level variable with the value of the next level variable. The main loop will run until nothing has been added to the next level.

Exercise!

Applying what we have just seen, implement a Breadth-First Search traverse function.

Don't continue until you have finished the exercise!

```python
def BFS(graph, start):
    """
    Perform Breadth-First Search of the graph starting from Vertex u.
    """
    # Prepare variables.
    # The only discovered vertex is the start vertex
    discovered = {start: None}
    # The only vertex in this level is the start vertex
    level = [start]
    while level:
        # Prepare a variable for the next level
        next_level = []
        # Iterate the vertices in this level
        for u in level:
            # Iterate the adjacents of this vertex
            for v in graph.get_adjacent_vertices(u):
                # If the adjacent is not yet in discovered
                if v not in discovered:
                    # Add the adjacent and the edge between them
                    discovered[v] = u
                    # Add also the adjacent to the next level
                    next_level.append(v)
        # When current level is exhausted, advance to the next level
        level = next_level
    return discovered
```

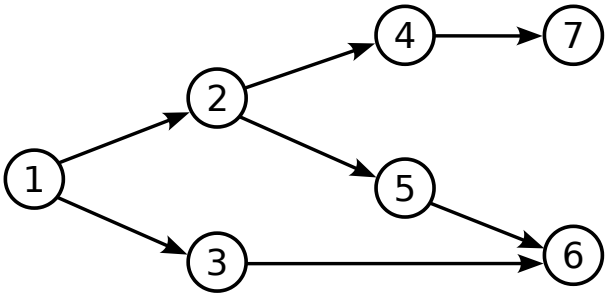
The BFS has some interesting properties, like the fact that the paths discovered for each vertex are the shortest paths (in terms of number of edges) that can be possible. Like with DFS, connectivity between vertices can also be determined with BFS.

The time complexity analysis of BFS is similar to DFS, with a runtime of $O(n+m)$ (being n the number of vertices reachable from starting vertex and m the number of edges incident to those vertices).

Directed Acyclic Graphs (DAG)

It's worth mentioning that in many applications, a certain type of graph is frequently encountered, the **Directed Acyclic Graph (DAG)**. This kind of graphs, as the name mentions, are directed, that is, that edges have direction and that means that given and edge `A --> B`, It is possible to go from A to B, but not from B to A (On the other hand, it is possible in a directed graph that exists at the same time an edge `B --> A`). **Acyclic** means there is no cycles in the graph. Given any vertex from the graph, there is no path that can take it back to the same vertex.

This kind of graph suits very well to situations where there are a series of tasks or activities that must be performed in order to achieve a goal.



In this image, if you want to reach to 2 or 3, you have first to visit 1. And you can reach 6 from 3 or from 5, i.e., there are multiple possible paths.

We are not going to see more of DAGs in this course though and it's left to the student as a self-study task.